

Files

↑ ↻ 📄 🔍

..

drive

sample_data

gradient_descent_plots.png

placement_dataset.csv

placement_predict_50k_adjuste...

[2] ✓ 23s

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

[4] ✓ 0s

import os
os.listdir('/content/drive/MyDrive')

['Copy of Session 7 Vocabulary (Synonyms and Antonyms) - Worksheet.docx',
'Screenshot_20250912_184215_Chrome.jpg',
'Screenshot_2025-12-28_162231.png',
'Copy of CSE WORKSHEET.gdoc',
'IMG-20260125-WA0002.jpg',
'Untitled form.gform',
'Untitled form (Responses).gsheet',
'2520030043.mp4',
'Colab Notebooks',
'CFAI python.ipynb',
'Classroom',
'122652ba-5dd1-472e-84b8-6956d98934fc.pdf',
'Learn-A-Thon_2025_certificate_akshita1115-gmail-com_de99c74b-116c-4fe2-a651-92acaec00db1.pdf']

```
import pandas as pd  
df = pd.read_csv('/content/placement_dataset.csv')
```

↑ ↓

▶

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

1. Load data

df = pd.read_csv("/content/placement_dataset.csv")

```

# -----
# 1. Load data
# -----
df = pd.read_csv("/content/placement_dataset.csv")

target_col = "CGPA"

# Drop rows where the target variable 'CGPA' is NaN
df.dropna(subset=[target_col], inplace=True)

# Drop the ID column and CGPA_Tier (a label derived directly from CGPA,
# so keeping it would leak the target) before selecting numeric features
# Updated drop_cols to only include columns present in the current DataFrame
drop_cols = [col for col in [target_col, "StudentID", "CGPA_Tier"] if col in df.columns]
feature_df = df.drop(columns=drop_cols).select_dtypes(include=[np.number])

# Some columns (Workshops, AptitudeTestScore, SoftSkillsRating,
# CodingTestScore, MockInterviewScore) have missing values -
# impute with the column median so gradient descent doesn't blow up on NaNs
feature_df = feature_df.fillna(feature_df.median())

X = feature_df.values
y = df[target_col].values.reshape(-1, 1)

# -----
# 2. Train/test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

# 3. Feature scaling (helps gradient descent converge)
# -----
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Add bias column (intercept term)
X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

# -----
# 4. Gradient Descent implementation
# -----
def gradient_descent(X, y, lr=0.01, n_iters=1000):
    m, n = X.shape
    theta = np.zeros((n, 1))
    losses = []

    for i in range(n_iters):
        y_pred = X @ theta
        error = y_pred - y

        # Mean Squared Error loss
        loss = (1 / (2 * m)) * np.sum(error ** 2)
        losses.append(loss)

        # Gradient of MSE w.r.t. theta
        gradient = (1 / m) * (X.T @ error)

        # Update rule

```

```

        theta -= lr * gradient

    if i % 100 == 0:
        print(f"Iteration {i:4d} | Loss: {loss:.4f}")

    return theta, losses

theta, losses = gradient_descent(X_train_b, y_train, lr=0.01, n_iters=1000)

# -----
# 5. Evaluate
# -----
y_pred = X_test_b @ theta

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nFinal weights (theta):")
print(theta.ravel())
print(f"\nTest MSE : {mse:.4f}")
print(f"Test R^2 : {r2:.4f}")

# -----
# 6. (Optional) Compare with sklearn's LinearRegression
# -----
from sklearn.linear_model import LinearRegression

sk_model = LinearRegression()
sk_model.fit(X_train, y_train)
sk_pred = sk_model.predict(X_test)

print(f"\nsklearn LinearRegression R^2: {r2_score(y_test, sk_pred):.4f}")

```

```
sk_pred = sk_model.predict(X_test)
```

```
print(f"\nsklearn LinearRegression R^2: {r2_score(y_test, sk_pred):.4f}")
```

```
Iteration    0 | Loss: 28.2456  
Iteration  100 | Loss:  4.1727  
Iteration  200 | Loss:  0.9475  
Iteration  300 | Loss:  0.5154  
Iteration  400 | Loss:  0.4575  
Iteration  500 | Loss:  0.4497  
Iteration  600 | Loss:  0.4487  
Iteration  700 | Loss:  0.4485  
Iteration  800 | Loss:  0.4485  
Iteration  900 | Loss:  0.4485
```

Final weights (theta):

```
[ 7.45513866  0.02584223 -0.07709395  0.02100797 -0.05290856]
```

Test MSE : 0.8066

Test R^2 : 0.0023

sklearn LinearRegression R^2: 0.0023

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, r2_score
```

```
# -----
```

```
# 1. Load data
```

```
"""
```

```

# -----
# 1. Load data
# -----
df = pd.read_csv("placement_predict_50k_adjusted.csv")

target_col = "CGPA"

sgpa_cols = [f"SGPA_Sem{i}" for i in range(1, 9)]
drop_cols = [c for c in [target_col, "StudentID", "CGPA_Tier"] + sgpa_cols if c in df.columns]
feature_df = df.drop(columns=drop_cols).select_dtypes(include=[np.number])

feature_df = feature_df.fillna(feature_df.median())

X = feature_df.values
y = df[target_col].values.reshape(-1, 1)

# -----
# 2. Train/test split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# 3. Feature scaling (helps gradient descent converge)
# -----
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

# 3. Feature scaling (helps gradient descent converge)
# -----
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Add bias column (intercept term)
X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

# -----
# 4. Gradient Descent implementation
# -----
def gradient_descent(X, y, lr=0.01, n_iters=1000):
    m, n = X.shape
    theta = np.zeros((n, 1))
    losses = []

    for i in range(n_iters):
        y_pred = X @ theta
        error = y_pred - y

        # Mean Squared Error loss
        loss = (1 / (2 * m)) * np.sum(error ** 2)
        losses.append(loss)

        # Gradient of MSE w.r.t. theta
        gradient = (1 / m) * (X.T @ error)

        # Update rule
        theta -= lr * gradient

```

```

        if i % 100 == 0:
            print(f"Iteration {i:4d} | Loss: {loss:.4f}")

    return theta, losses

theta, losses = gradient_descent(X_train_b, y_train, lr=0.01, n_iters=1000)

# -----
# 5. Evaluate
# -----
y_pred = X_test_b @ theta

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nFinal weights (theta):")
print(theta.ravel())
print(f"\nTest MSE : {mse:.4f}")
print(f"Test R^2 : {r2:.4f}")

# -----
# 6. (Optional) Compare with sklearn's LinearRegression
# -----
from sklearn.linear_model import LinearRegression

sk_model = LinearRegression()
sk_model.fit(X_train, y_train)
sk_pred = sk_model.predict(X_test)

```



```
sk_model = LinearRegression()
sk_model.fit(X_train, y_train)
sk_pred = sk_model.predict(X_test)

print(f"\nsklearn LinearRegression R^2: {r2_score(y_test, sk_pred):.4f}")

# -----
# 7. Plots
# -----
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# (a) Gradient descent loss curve – shows convergence over iterations
axes[0].plot(range(len(losses)), losses, color="steelblue")
axes[0].set_title("Gradient Descent: Loss vs Iterations")
axes[0].set_xlabel("Iteration")
axes[0].set_ylabel("MSE Loss (1/2m * sum of squared error)")
axes[0].grid(alpha=0.3)

# (b) Actual vs Predicted CGPA on the test set
axes[1].scatter(y_test, y_pred, alpha=0.15, s=8, color="darkorange")
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
axes[1].plot(lims, lims, color="black", linestyle="--", linewidth=1, label="Perfect prediction")
axes[1].set_title("Actual vs Predicted CGPA (Test Set)")
axes[1].set_xlabel("Actual CGPA")
axes[1].set_ylabel("Predicted CGPA")
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
```

```
... Iteration    0 | Loss: 27.1394
      Iteration  100 | Loss: 3.5957
      Iteration  200 | Loss: 0.5039
      Iteration  300 | Loss: 0.0894
      Iteration  400 | Loss: 0.0338
      Iteration  500 | Loss: 0.0263
      Iteration  600 | Loss: 0.0252
      Iteration  700 | Loss: 0.0251
      Iteration  800 | Loss: 0.0250
      Iteration  900 | Loss: 0.0250
```

Final weights (theta):

```
[ 7.29885264e+00  1.51572915e-01  6.19255015e-02  9.18994042e-02
  6.11528698e-02  9.21034270e-02  3.59116610e-02  1.42812012e-01
  1.53586067e-01  1.62139076e-01  1.37183398e-01  2.54455045e-02
 -3.15083847e-04]
```

Test MSE : 0.0502

Test R² : 0.9508

sklearn LinearRegression R²: 0.9508

Saved plots to gradient_descent_plots.png

Saved plots to gradient_descent_plots.png

